Cloud Computing Project Plan

Hire Lens

Mohammed Zaid Shaikh Piotr Bartosiewicz Krzysztof Krawiec

April 2026

Contents

1 Brief Description

We propose a Telegram-first cloud platform that helps HR teams analyze GitHub profiles of candidates and compare them against job descriptions. The bot will generate a structured profile review from a GitHub username or profile URL, summarize public activity, highlight technical strengths and weak signals, and later support linked profiles such as GitHub, LinkedIn, and personal websites.

The first version focuses on safe, explainable automation: the bot will retrieve public GitHub information, run scoring and summarization workflows, and present a concise report to the HR user. In later iterations, the system can include AI-assisted compatibility scoring between a candidate profile and a job description.

A small web dashboard may be added later for admin and analytics tasks, but the Telegram bot remains the primary frontend.

To avoid dependence on paid AI APIs, the analysis layer can be powered by a self-hosted open-source model such as Qwen. The model would run behind an internal inference service and be called only by our backend, which keeps prompts, data, and costs under our control.

1.1 Recommended Hosting Choice

The easiest beginner-friendly hosting setup on Azure is:

- **Azure Functions**: Telegram webhook, OAuth callbacks, report endpoints, and access-control logic.
- **Azure Container Apps**: the self-hosted model-inference service, exposed only internally to the backend.
- **Azure Static Web Apps**: a small admin dashboard if we need one.
- **Azure Cosmos DB, Blob Storage, and Service Bus**: managed data, files, and background jobs.

This avoids AKS, avoids always-on virtual machines for the application layer, and keeps the infra simple enough for beginners while still being realistic for the course.

2 Main Features

1. **Telegram profile review**: HR sends a GitHub username or link and gets a structured evaluation.
2. **Job description comparison**: HR can paste a job description and receive a compatibility summary.
3. **Candidate shortlisting**: save candidate reports, tags, and follow-up notes.
4. **Public-data enrichment**: gather public GitHub signals such as repositories, languages, commits, followers, stars, and contribution patterns.
5. **Access control**: limit usage by subscription tier and organization role.
6. **Notification scheduling**: send reminders, report updates, and saved-search alerts.
7. **Payment readiness**: Stripe can be added later for premium access plans.

3 Typical User Roles

- **HR Recruiter**: searches and reviews candidates, compares profiles with job descriptions, and saves reports.
- **Team Lead/Manager**: reviews shortlisted candidates and monitors report quality.
- **Platform Admin**: manages user access, usage limits, and billing tiers.

4 System Architecture

4.1 Service Overview

- **Frontend**: Telegram bot as the primary UI.
- **Optional Admin UI**: lightweight React dashboard for analytics and moderation.
- **API Layer**: Azure Functions exposing REST endpoints for bot actions and admin operations.
- **Auth**: Google/Facebook sign-in through OAuth, with no password storage.
- **Session Layer**: server-side session or short-lived token session store.
- **Primary Data**: Azure Cosmos DB for users, candidate reports, access rules, and usage records.
- **Unstructured Assets**: Azure Blob Storage for report exports, cached snapshots, and attachments.
- **Async Work**: Azure Service Bus and scheduled Functions for profile refresh, notifications, and ranking jobs.
- **Observability**: Application Insights and Azure Monitor with alerts.

4.2 Microservice-style Functions

- `bot-gateway-service`: receives Telegram updates and routes commands.
- `auth-service`: handles OAuth sign-in, session creation, and access checks.
- `profile-analysis-service`: fetches public GitHub data and creates profile summaries.
- `model-inference-service`: runs a self-hosted open-source model such as Qwen inside Azure Container Apps.
- `job-match-service`: compares candidate profiles with job descriptions using rule-based logic plus model output.
- `report-service`: stores and retrieves analysis reports.
- `notification-service`: scheduled reminders and saved-search alerts.
- `billing-service`: premium plan logic and Stripe integration for later versions.

5 API Design

Representative REST endpoints:

- `POST /api/v1/auth/oauth/start`
- `POST /api/v1/auth/oauth/callback`
- `POST /api/v1/bot/webhook`
- `POST /api/v1/profiles/analyze`
- `POST /api/v1/models/infer`
- `POST /api/v1/match/compare`
- `GET /api/v1/reports/reportId`
- `GET /api/v1/users/me/usage`
- `POST /api/v1/admin/limits`

6 Storage Characteristics

Store	Type	Consistency	RW Pattern	Data Scope
Azure Cosmos DB	NoSQL, semi-structured documents	Tunable consistency, strong for critical records	Heavy read/write	Users, sessions, reports, usage limits, billing flags
Azure Blob Storage	Unstructured objects	Strong read-after-write for object access	Read-heavy with periodic writes	Report exports, cached payloads, attachments
Azure Table Storage or SQL (optional)	Structured metadata	Strong for transactional data, depending on choice	Moderate read/write	Audit logs, subscriptions, premium access records

Table 1: Planned storage profile and access patterns.

7 Terraform Plan

Infrastructure will be provisioned with Terraform and kept reproducible from the beginning:

- `modules/frontend` for Azure Static Web Apps and the optional dashboard deployment.
- `modules/functions` for Azure Functions, Telegram webhooks, OAuth callbacks, and scheduled jobs.
- `modules/auth` for OAuth and session-related resources.
- `modules/data` for Cosmos DB, Blob Storage, and optional SQL/Table resources.
- `modules/messaging` for Service Bus queues and topics.
- `modules/model` for Azure Container Apps hosting the self-hosted inference service.

- `modules/monitoring` for dashboards and alerting.
- `environments/dev` and `environments/prod` for environment-specific variables.

Minimum deliverable: one `terraform apply` that creates a working dev environment with bot backend, internal model service, database, storage, async messaging, and monitoring.

8 Diagrams

The following diagrams are included in the submission package.

8.1 Architecture Diagram

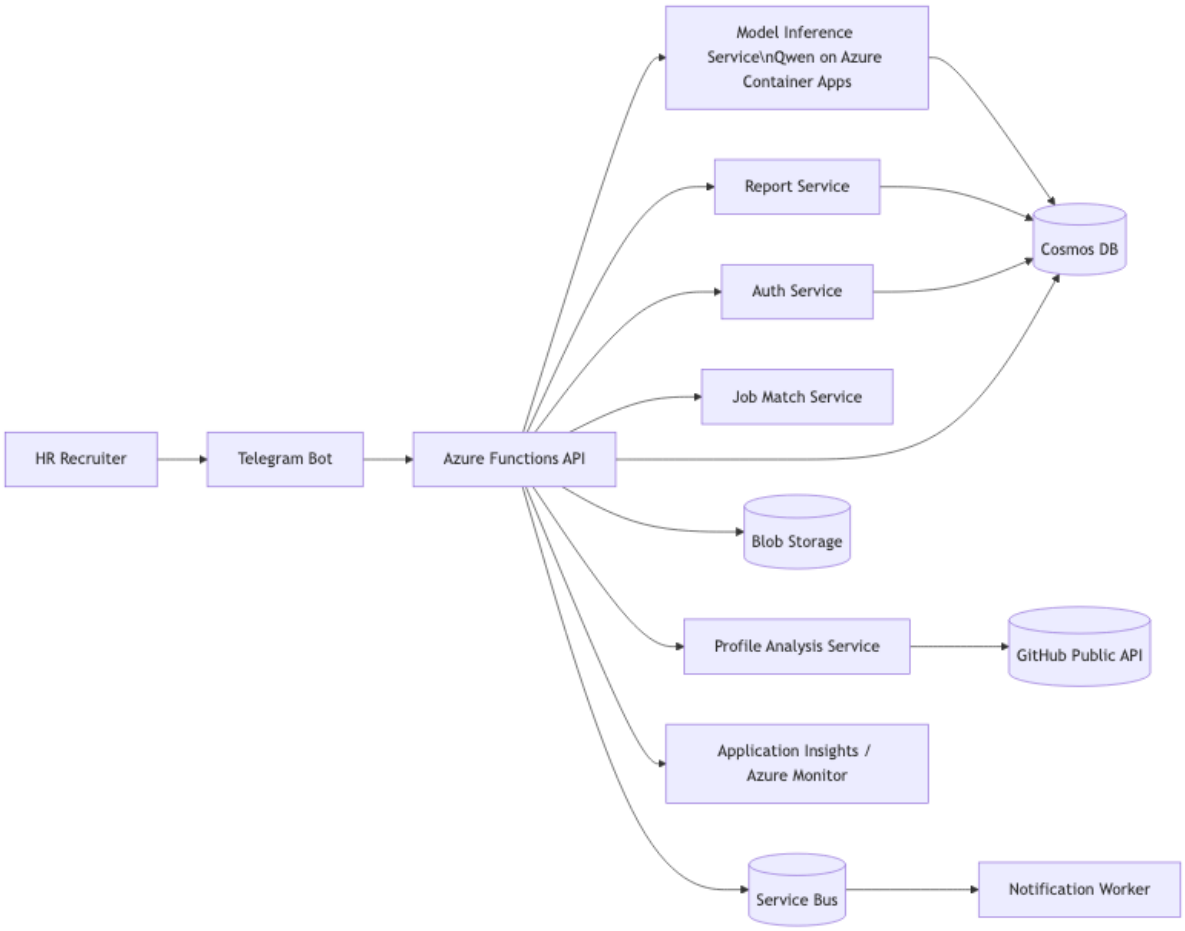

Figure 1: System architecture diagram.

8.2 Request Flow Diagram

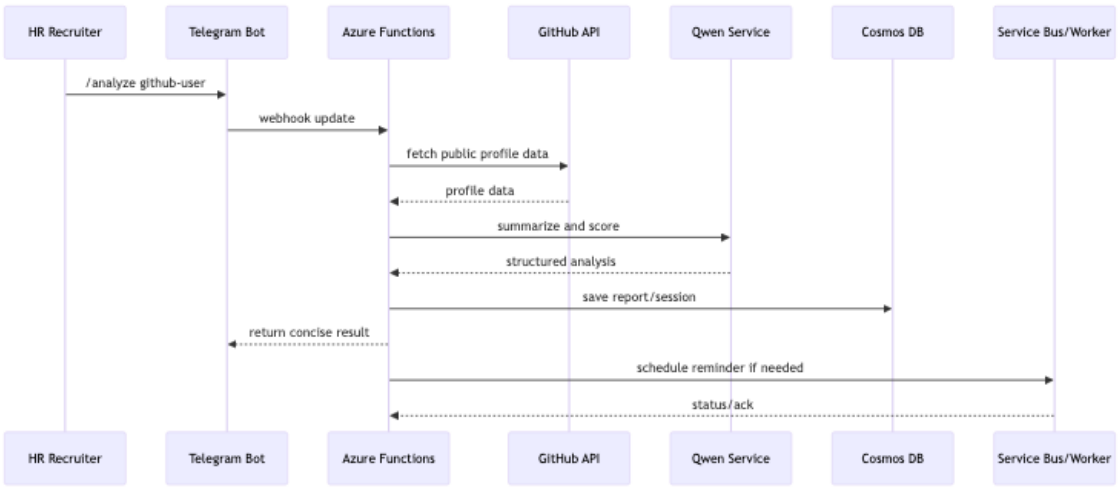

Figure 2: Request flow diagram.

8.3 Storage Diagram

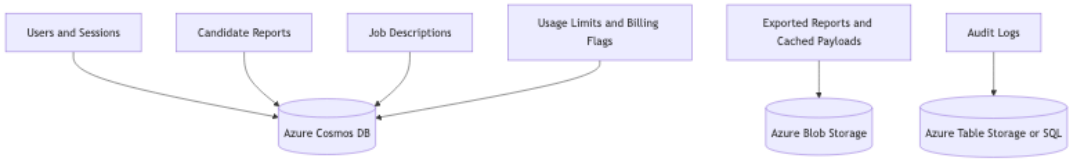

Figure 3: Storage layout diagram.

9 Large-Scale Assumption

The architecture assumes thousands of HR users and repeated candidate reviews per day, with spikes during hiring campaigns. Scaling strategy:

- Stateless Azure Functions that scale on demand.
- Queue-based processing for profile analysis and enrichment jobs.
- Azure Container Apps for the model endpoint so inference cost is predictable and data stays inside our system.
- Caching of frequent GitHub responses and report snapshots.
- Rate limiting and access tiers to protect public API usage and cloud costs.

10 Service Level Agreement (SLA)

This Service Level Agreement defines the expected reliability, performance, and operational guarantees of the Hire Lens platform deployed in a cloud environment.

10.1 Service Scope

The SLA applies to the following system components:

- Telegram bot interaction layer (bot-gateway-service)
- Backend API (Azure Functions)
- Profile analysis and job matching services
- Internal model inference service
- Data storage (Cosmos DB, Blob Storage)

External dependencies such as GitHub API and OAuth providers are excluded from strict guarantees but are considered in best-effort availability.

10.2 Service Level Indicator (SLI)

A Service Level Indicator we defined as the percentage of successful HTTP requests relative to the total number of HTTP requests over a specified time period. A request is considered successful if it returns a response with status code 200-399 within the acceptable response time.

10.3 Service Level Objectives (SLO)

The system defines the following Service Level Objectives based on the SLI metrics:

- **Availability SLO:** 99.0% of requests are successful over a monthly period
- **Latency SLO:** 95% of requests are served within 2 seconds
- **Error Rate SLO:** less than 1% of requests result in errors (HTTP 5xx or timeout)

10.4 Performance (Latency Objectives)

The system defines the following response time objectives:

- **API response time:** below 2 seconds
- **Maximum response time:** 5 seconds
- **Model inference response time:** below 2 minutes (asynchronous processing allowed)

Long-running operations such as profile analysis may be processed asynchronously using queues.

10.5 Error Rate

- **Target error rate:** less than 1% of total requests
- Errors include HTTP 5xx responses and timeouts

10.6 Security

- All communication is secured via HTTPS
- OAuth-based authentication (no password storage)
- Access control enforced per user role and subscription tier

10.7 Monitoring and Observability

The system uses cloud-native monitoring tools:

- Azure Monitor and Application Insights for logs and metrics
- Alerting on:
  - high error rate
  - increased latency
  - service downtime

10.8 Maintenance and Downtime

- Planned maintenance will be scheduled during low-traffic periods
- Users will be notified in advance when possible
- Planned downtime is excluded from SLA calculations

10.9 Violation Handling

In case the SLA objectives are not met:

- The incident is recorded and analyzed
- Root cause analysis is performed
- Improvements are proposed for future iterations

11 Delivery Plan and Team Split

11.1 Submission Scope

- Final project idea, features, user roles, and high-level architecture.
- Diagram sources in Mermaid format for the final submission package.
- API design, storage profile, Terraform setup, and SLA/SLO/SLI definitions.
- Live-demo plan with AI-assisted comparison and usage limits.

11.2 Proposed Responsibilities

- **Mohammed Zaid Shaikh**: Telegram bot flow, UX, and command handling.
- **Piotr Bartosiewicz**: Azure Functions, GitHub analysis logic, and API contracts.
- **Krzysztof Krawiec**: Terraform, monitoring, deployment automation, and load testing.

12 Risk and Mitigation

- **Risk**: GitHub public API rate limits.
- **Mitigation**: caching, queueing, and conservative refresh intervals.
- **Risk**: AI output quality or hallucination.
- **Mitigation**: structured prompts, explainable scoring, and fallback rule-based summaries.
- **Risk**: GPU hosting cost for the open-source model.
- **Mitigation**: start with a smaller quantized Qwen model on Azure Container Apps, keep inference behind caching and rate limits, and move to a GPU-only option only if performance is not enough.
- **Risk**: OAuth setup complexity for beginners.
- **Mitigation**: start with one provider, then add the second provider once the flow is stable.
- **Risk**: Costs increase with external API calls and network traffic.
- **Mitigation**: use the cheapest practical Azure region, limit retries, and add usage caps per tier.

13 Conclusion

This project keeps the implementation practical while still looking like a serious cloud system: a Telegram-first product, Azure managed services, Terraform-based infrastructure, and a clear path from rule-based analysis to AI-assisted hiring support.